

Reflexión Personal

¿Qué fue lo más difícil de entender?

Lo que más me costó fue la simplificación de proposiciones mediante las leyes lógicas. No era complicado memorizar los nombres, el problema era saber *cuándo y en qué orden aplicar cada ley* para llegar a una expresión más simple sin cometer errores en el camino.

Con las tablas de verdad uno puede verificar un resultado paso a paso, pero con las leyes hay que tener claro qué forma tiene la expresión antes de tocarla. Por ejemplo, aplicar De Morgan en el lugar equivocado o confundir la equivalencia del condicional con la contraposición generaba resultados incorrectos que eran difíciles de detectar después. Fue un tema que requirió más práctica que los demás para que empezara a sentirse natural.

¿Qué tema comprendí mejor?

Las tablas de verdad fueron el tema que mejor entendí. Desde el principio me resultó claro el procedimiento: definir las variables, listar todas las combinaciones posibles de valores y evaluar la expresión columna por columna.

Lo que más me ayudó fue entender que las tablas tienen una lógica estructural fija: con n variables siempre hay 2^n filas, y el orden de V y F en cada columna sigue un patrón que no cambia. Una vez que eso quedó claro, construir tablas para expresiones largas dejó de ser confuso y pasó a ser simplemente un proceso metódico. También fue satisfactorio usarlas para verificar si algo era tautología, contradicción o contingencia, porque el resultado se ve directamente sin necesidad de razonar de forma abstracta.

¿Cómo puedo aplicar la lógica en mi carrera?

En Ingeniería de Computación la lógica proposicional no es solo un tema teórico, es una herramienta que aparece de forma constante en distintas áreas de la carrera.

La aplicación más directa está en la **programación**. Cada vez que se escribe una condición *if*, un bucle o una validación de datos, se está trabajando con proposiciones y conectores lógicos sin necesidad de escribirlos con símbolos. Saber si una condición compuesta puede simplificarse, o si dos condiciones distintas son equivalentes, ayuda a escribir código más limpio y menos propenso a errores.

También tiene un rol clave en el **diseño de circuitos digitales**. Las compuertas lógicas AND, OR y NOT son exactamente los operadores $\wedge$, $\vee$ y $\neg$. Un circuito combinacional se puede describir con una expresión proposicional y optimizarse usando las mismas leyes que se aplican en lógica: De Morgan, absorción, distribución. Esto reduce el número de compuertas necesarias, lo que se traduce en circuitos más eficientes.

En **bases de datos**, las consultas con condiciones múltiples combinan proposiciones con AND, OR y NOT, y entender cómo se evalúan esas condiciones permite construir consultas correctas y eficientes. En **ciberseguridad**, las reglas de acceso, firewalls y sistemas de autenticación se modelan con condiciones lógicas similares a las que trabajamos en este tema.

En general, la lógica proposicional me da una forma estructurada de analizar cualquier problema con condiciones: descomponerlo en partes simples, expresarlo con precisión y razonar sobre él sin ambigüedades. Eso es exactamente lo que se necesita al diseñar sistemas, escribir algoritmos o depurar código.